

Laboratory 4: UVM-Based Verification of a DCT Coprocessor with an AXI Interface

Github repository: <https://github.com/rgenovese-uab/aipcd-lab4/>

Objective:

Students will develop a UVM testbench that verifies an RTL implementation of a DCT coprocessor. The testbench will interact with the DUT via an AXI VIP to:

- Write an 8×8 input block to the coprocessor.
- Initiate the DCT operation by writing to the CTRL register.
- Poll the STATUS register until the operation is complete.
- Read back the output block.
- Compare the DUT's output with the expected results produced by the reference C model.

Goals:

- Develop a UVM testbench that drives AXI transactions to the DUT, using a Mentor Questa VIP.
- Verify that the DCT RTL module correctly transforms an 8×8 image block by comparing its output to a golden reference from the C model.

This approach provides a hands-on verification lab that builds on your previous work, challenges students with a new protocol (AXI), and reinforces the concepts of transaction-based verification and modular testbench design.

Key Tasks for Students:

1. Understand the DUT Interface:

- Review the memory-mapped registers (CTRL, STATUS, in_block, out_block).
- Understand how the AXI transactions are used to trigger the DCT operation and retrieve data.

2. Develop a UVM Testbench:

○ AXI VIP Integration:

Use an AXI VIP that can drive read/write transactions. The VIP should support:

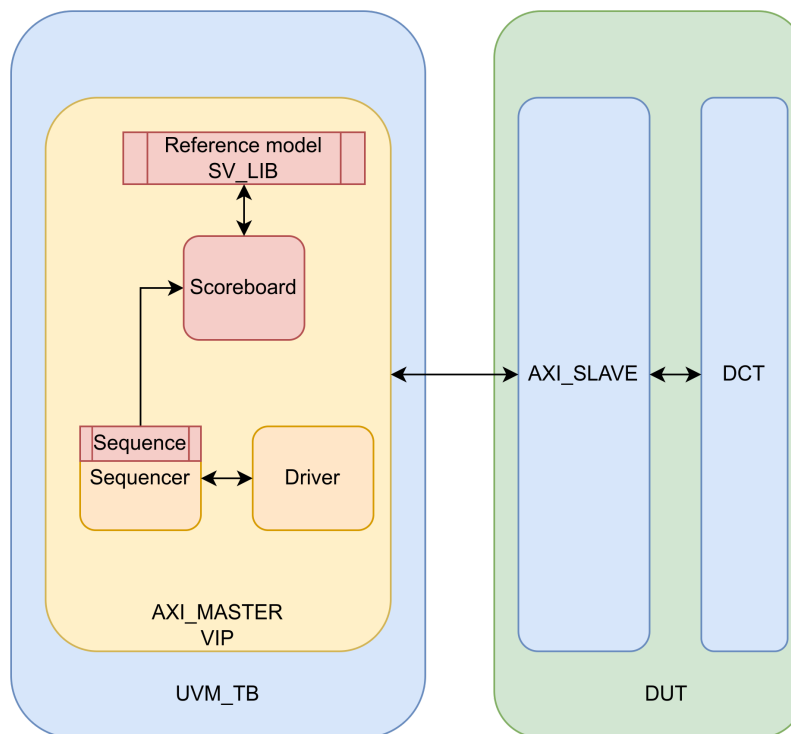
- AXI write transactions to send input data and the start command.
- AXI read transactions to poll the STATUS register and retrieve the output block.

○ Sequence:

Create a series of AXI sequences to generate the corresponding AXI transactions in order to drive the DCT and read its results.

- **Checker:**
Implement a Scoreboard that captures the DUT's output and compares these outputs with the expected values computed by the provided C function. This could involve instantiating a simple C model as a reference (or pre-calculating golden outputs) and comparing on a transaction basis.

How does the DCT coprocessor work?



```

void dct_hw(float in_block[8][8], float out_block[8][8])
{
    Uint* p = 0x.... // base address of DCT coprocessor
    Memcpy(&p[2], in_block, 64*4);
    p[0] = START;
    while (p[1] != DONE){ // wait}
    Memcpy(out_block, &p[64+2], 64*4);
}

```

The `dct_hw` function is a software-level routine that simulates how a processor would interact with the DCT coprocessor via the AXI interface. In the actual hardware design, the DCT module is implemented as an **AXI slave**, which means it responds to transactions—such as writes for

input data and control signals, and reads for output data—initiated by an AXI master. The AXI VIP acts as the **AXI master**, generating those transactions toward the DCT module.

The DUT (the DCT RTL module with an AXI slave interface) would be instantiated in the top-level module. The verification IP (AXI master) connects to this DUT, driving transactions that mimic the behavior of the `dct_hw` function. Essentially, the `dct_hw` function shows what should happen at the software level, while your UVM-based testbench will stimulate the DUT via AXI transactions as an AXI master communicating with an AXI slave.

The AXI master should mimic the sequence performed by the `dct_hw` function. In other words, it needs to perform the following AXI transactions:

1. **Write Input Data:**
 - Perform a burst write transaction to the input block memory starting at address 0x8.
 - This transaction transfers the 64 input words (each 64-bit) that represent the 8×8 block data.
2. **Issue Start Command:**
 - Execute a single write transaction to the CTRL register at address 0x0, writing the “START” command.
 - This tells the DUT (the DCT coprocessor) to begin processing the input data.
3. **Poll Status:**
 - Perform repeated (or burst) read transactions on the STATUS register at address 0x4.
 - Continue polling until the returned value indicates that processing is complete (i.e., the “DONE” flag is set).
4. **Read Output Data:**
 - Once the status indicates completion, perform a burst read transaction from the output block memory.
 - The output block is located starting at address $0x8 + (64 \times 4)$ (i.e., after the input block), and the master reads back the 64 output words representing the transformed data.

These transactions—burst writes for input data, a control write to start processing, polling the status, and burst reads for output data—form the complete AXI transaction sequence your master must send to the DUT.

Overview of AXI4 Protocol

AXI4 (Advanced eXtensible Interface) is a high-performance, memory-mapped protocol that is part of the AMBA (Advanced Microcontroller Bus Architecture) family from ARM. It is widely used in SoCs (System on Chip) to connect processors, accelerators, and memory subsystems efficiently.

AXI4 Protocol and Its Key Features

AXI4 provides high-bandwidth, low-latency communication with support for burst transactions and multiple outstanding transactions. The protocol is split into **five independent channels**, each with its own handshake mechanism:

1. Write Address Channel (AW)

- Sends the address of a write transaction.
- Key signals:
 - **AWADDR**: Address of the write transaction.
 - **AWPROT**: Protection attributes.
 - **AWLEN**: Number of data beats in a burst.
 - **AWSIZE**: Size of each data beat.
 - **AWBURST**: Burst type (e.g., INCR, WRAP, FIXED).
 - **AWVALID**: Master asserts when the address is valid.
 - **AWREADY**: Slave asserts when it is ready to receive the address.

2. Write Data Channel (W)

- Transfers the actual data to be written.
- Key signals:
 - **WDATA**: Data being transferred.
 - **WSTRB**: Byte-enable signals to indicate which bytes are valid.
 - **WLAST**: Indicates the last data beat in a burst.
 - **WVALID**: Master asserts when data is valid.
 - **WREADY**: Slave asserts when ready to accept data.

3. Write Response Channel (B)

- Provides acknowledgment of the write transaction.
- Key signals:
 - **BRESP**: Indicates write response status (OKAY, SLVERR, DECERR).
 - **BVALID**: Slave asserts when response is valid.
 - **BREADY**: Master asserts when it is ready to receive the response.

4. Read Address Channel (AR)

- Sends the address of a read transaction.
- Key signals:
 - **ARADDR**: Address of the read transaction.

- **ARPROT**: Protection attributes.
- **ARLEN**: Number of data beats in a burst.
- **ARSIZE**: Size of each data beat.
- **ARBURST**: Burst type.
- **ARVALID**: Master asserts when the address is valid.
- **ARREADY**: Slave asserts when it is ready to receive the address.

5. Read Data Channel (R)

- Transfers the read data from the slave to the master.
- Key signals:
 - **RDATA**: Data being transferred.
 - **RRESP**: Indicates read response status.
 - **RLAST**: Indicates the last data beat in a burst.
 - **RVALID**: Slave asserts when data is valid.
 - **RREADY**: Master asserts when it is ready to accept data.

Handshaking Mechanism

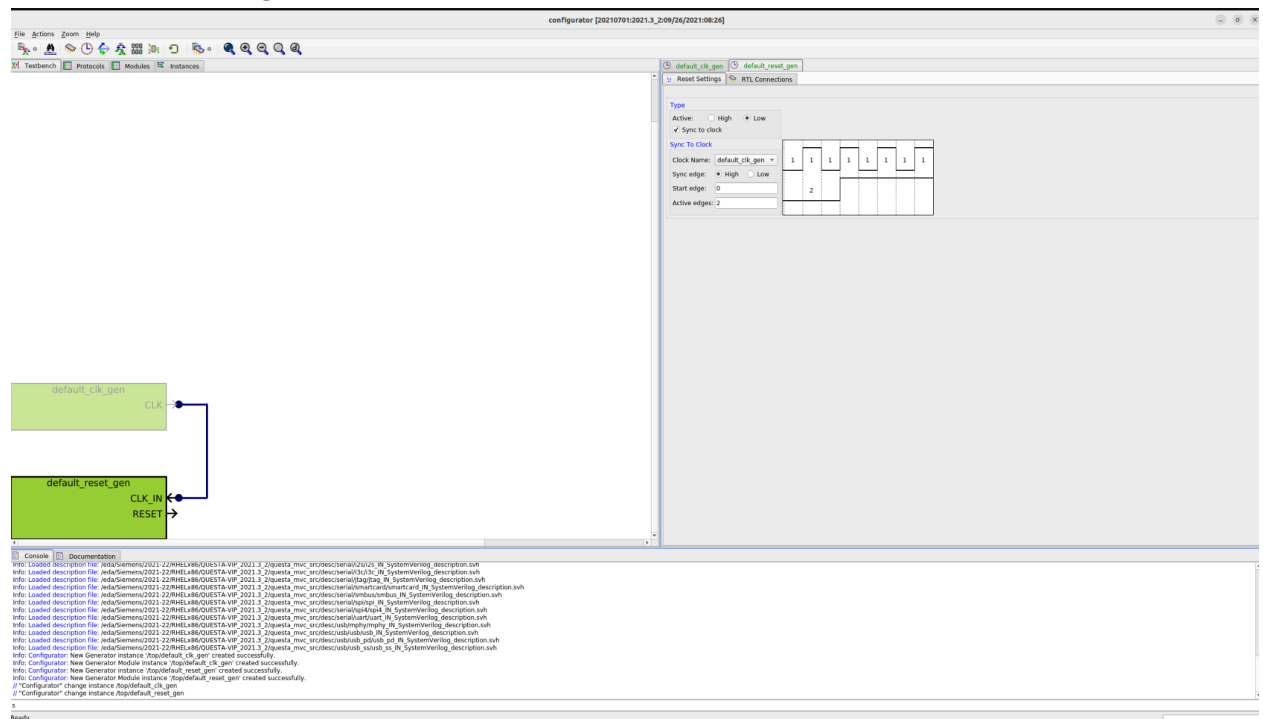
All channels use **valid (XVALID)** and **ready (XREADY)** signals to establish communication between the master and the slave. A transaction proceeds only when both the master and the slave assert their respective valid and ready signals.

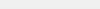
Environment Setup

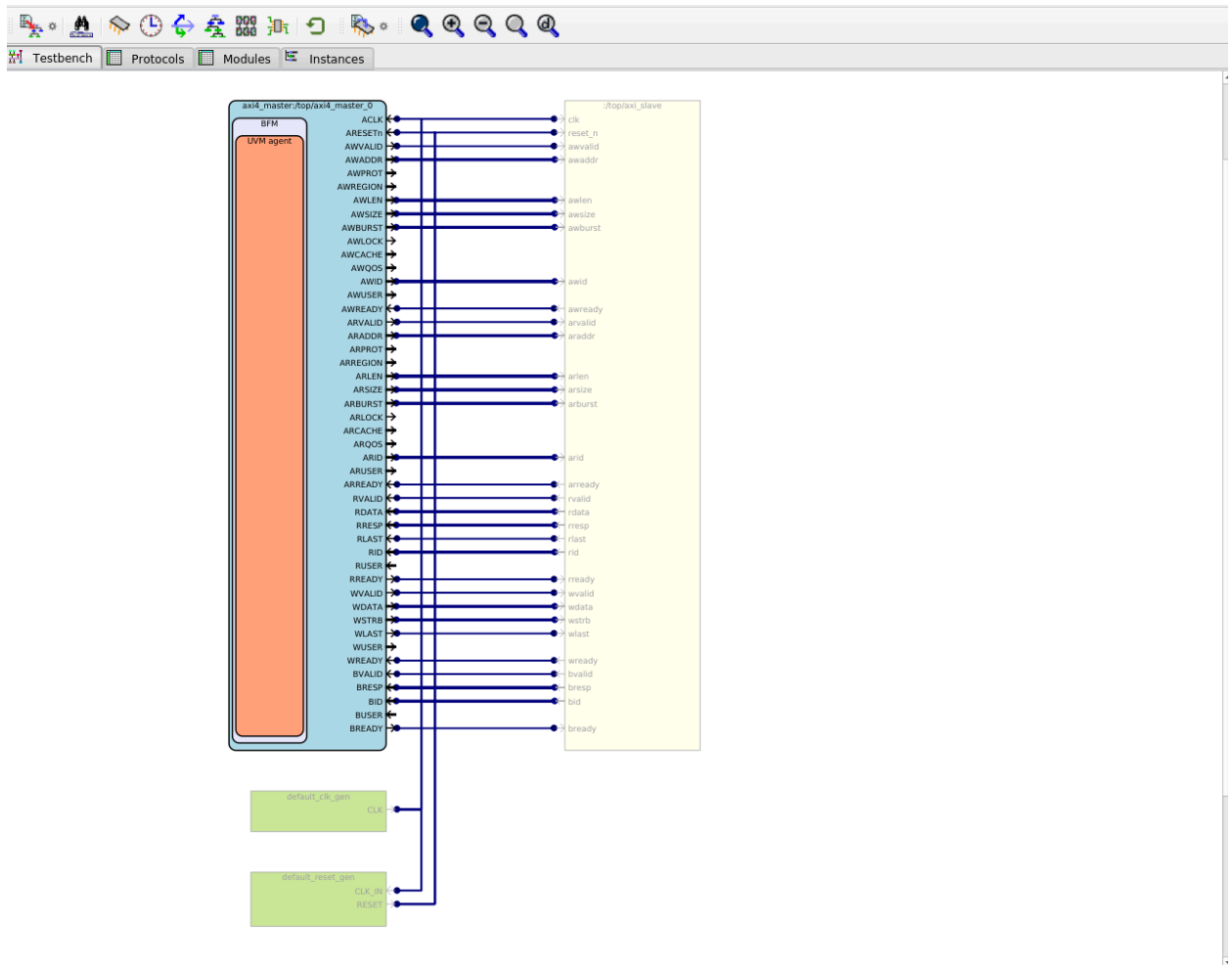
1. Make sure the following environment variables are properly set:
 - 1.1. QVIP_HOME=/tools/Siemens_EDA/questasim/2021.3/
 - 1.2. QUESTA_MVC_HOME=/tools/Siemens_EDA/questasim/2021.3/
 - 1.3. PATH="\${PATH}:\${QVIP_HOME}/bin"
2. Clone the repository (<https://github.com/rgenovese-uab/aipcd-lab4/>) and cd its folder.

UVM Testbench creation using qvip

- ### 3. Launch **qvip_configurator**



4. Add an AMBA AXI4 Master VIP instance ()
5. Import the provided RTL module (axi_slave_64bits.sv) as DUT ()
6. Connect the modules according to the AXI4 Protocol. Don't forget to connect the clock and reset signals.



7. Create an Address Map () with the following configuration:

- CTRL at 0x0, size 8 bytes
- STATUS at 0x8, size 8 bytes
- IN_BLOCK at 0x10, size 512 bytes
- OUT_BLOCK at 0x210, size 512 bytes

AXI	CTRL	0x0
	STATUS	0x8
	In_block[64]	0x10
	out_block[64]	0x210

Create / Edit Address Map

DCT

Name

Add Delete Copy

Map Entries

Kind: MAP_NORMAL

Name: OUT_1

Id: 0

Domain: MAP_NS

Region: 4'h0

Addr: 64'h410

Size: 64'h1000

Mem: MEM_NORMAL

Prot: MAP_NORM_SEC_DATA

Add Delete

64'h0

64'h8

64'h10

64'h210

64'h410

"CTRL"

"STATUS"

"IN_BLOCK"

"OUT_BLOCK"

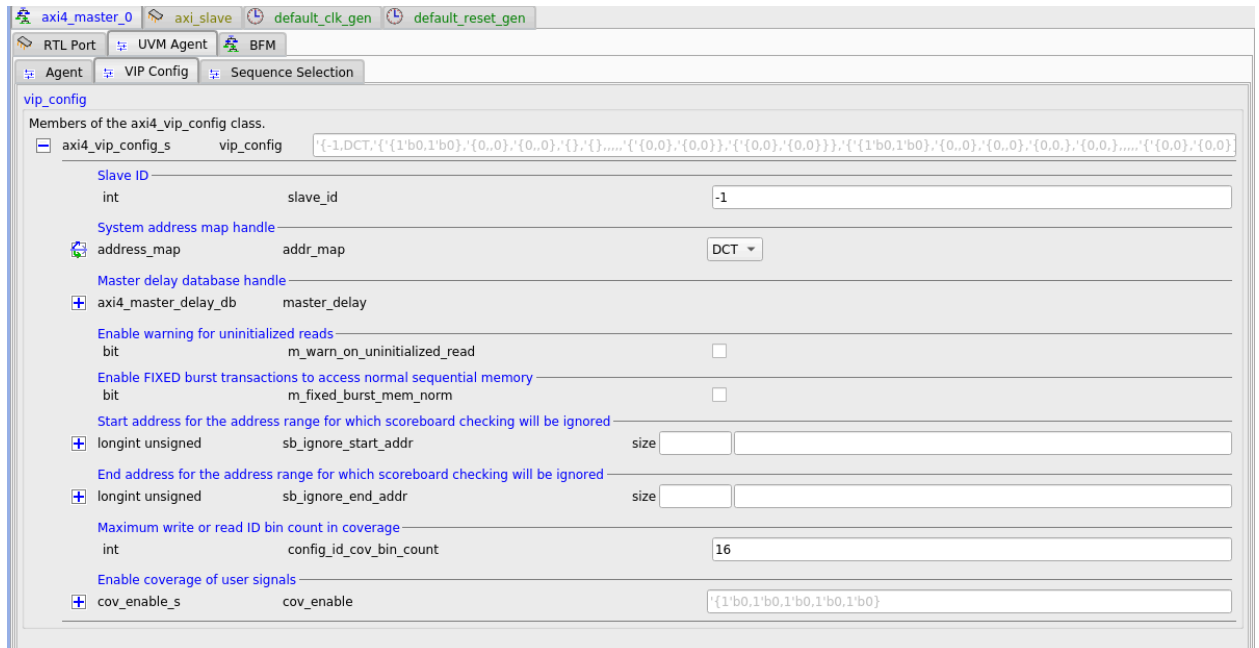
Additional

Address mask

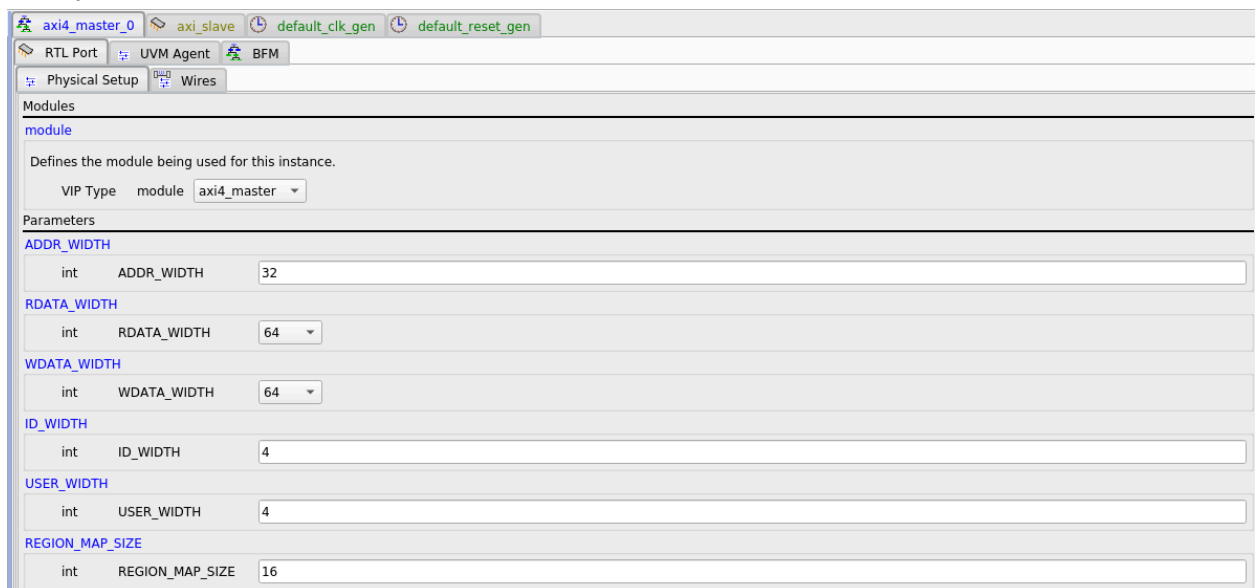
longint unsigned addr_mask 64'hFFF

Cancel

8. Assign the Address Map to the AXI4 Master VIP



9. Modify the AXI4 Master VIP data width to 64



10. Assign 4 sequences to the AXI 4 Master VIP. Since the DCT coprocessor will support AXI LITE (no burst operations) the necessary sequences are:
 - 10.1. A write transaction to write the 64 input values to the IN_BLOCK, which will be executed 64 times.
 - 10.2. A write transaction to write the CTRL register and indicate to start the DCT operation
 - 10.3. A read transaction to read the STATUS register, that should be executed multiple times until the read value indicates the DCT process has been finished.

- 10.4. A read transaction to read the 64 output values from the OUT_BLOCK, which will be executed 64 times.

The screenshot shows the 'Sequence Selection' tab in the qvip_configurator. The 'Selected Sequences' list contains the following entries:

Instance / Action	Sequence
axi4_master_0	axi4lite_wr_data_deparam_seq
axi4_master_0	axi4lite_wr_data_deparam_seq
axi4_master_0	axi4lite_rd_data_deparam_seq
axi4_master_0	axi4lite_rd_data_deparam_seq

Below the list is a table with the following columns: Name, Description.

Name	Description
# Atomic Access Sequences	
axi4_excl_rw_deparam_seq	Executes exclusive read and write transactions sequence
# Concurrent Sequences	
axi4_wr_os_diff_id_deparam_seq	Executes concurrent write transactions with interleaved data
# Burst Type Sequences	
axi4_incr_rd_deparam_seq	Executes increment read transactions
axi4_incr_wr_deparam_seq	Executes increment write transactions
axi4_fixed_rd_deparam_seq	Executes fixed read transactions
axi4_fixed_wr_deparam_seq	Executes fixed write transactions
axi4_wrap_rd_deparam_seq	Executes wrap read transactions
axi4_wrap_wr_deparam_seq	Executes wrap write transactions
# Delay Sequences	
axi4_wr_addr2data_all_dly_deparam_seq	Executes write transactions with address to data delay and all other write delays
axi4_rd_rvalid2rdy_dly_deparam_seq	Executes read transactions with read data valid to ready delay
# AXI4Lite Sequences	
axi4lite_rd_data_deparam_seq	Executes basic read transaction
axi4lite_wr_data_deparam_seq	Executes basic write transaction
axi4lite_wr_addr2data_all_dly_deparam_seq	Executes write transactions with address to data delay and all other write delays
axi4lite_rd_rvalid2rdy_dly_deparam_seq	Executes read transactions with read data valid to ready delay

11. Save the commands (File -> Save Commands). The generated cmd file must be in the final deliverable.
12. Generate the current testbench (). This should create a **top_dir** folder with the generated files.
13. Close **qvip_configurator**.

Stimulus/Sequences modification

14. The files to work with are inside the **top_dir/uvvm_tb** folder. Make sure to:

- 14.1. Check the generated Makefile. It should correctly include the path to the provided `axi_slave.sv` file. Check the available make targets. Modify according to your needs:
 - 14.1.1. Remove the `-c` from the `sim` target, so GUI opens when doing a `make sim`;
 - 14.1.2. Add the `+acc` flag to the `vlog` arguments (under the `hdl_sv` target).
 - 14.1.3. Check that the DUT path is correct.
 - 14.1.4. Add the `dct_compute.sv` file (with its correct path).
- 14.2. Modify the Address Map mask (in `top_dir/uvm_tb/top_env_config.svh`) to allow unaligned accesses (ie: `DCT.addr_mask = 64'h3;`).
- 14.3. Added sequences are created and executed in **`top_dir/uvm_tb/top_example_vseq.svh`**
 Read the documentation for each sequence and modify them according to the addresses/values you need to write/read. The polling of the STATUS register can be done in parallel to the write sequences until the process is DONE.

These are the for sequences you need to program:

- **`axi4lite_wr_data_deparam_seq axi4_master_0_seq_0`**: sequence to write 64 real (8 bytes) numbers, starting from address `0x10` (`IN_BLOCK`). It's documentation can be found at: [\\$QVIP_HOME/docs/html/axi4/files/axi4/sequence_lib/stimuli/axi4lite/deparam/data_transfer/wr_data_deparam-svh.html](#)
 The sequence fields to modify are:
 - `addr`: indicating the starting address of the burst.
 - `wr_data`: a byte array containing the bytes corresponding to one of the 64 8-byte values.
 You could randomly create 64 real (IEEE 754 Double-Precision) values and then use the `$realtobits` SystemVerilog function to convert them into a `logic[63:0]`, which can later be passed to the `wr_data` bytes.
 This sequence should be executed (ie: call the `start()` method) 64 times, one for each value.
- **`axi4lite_wr_data_deparam_seq axi4_master_0_seq_1`**: a basic write transaction to write into the CTRL register (`0x0`) and indicate the DCT to start the operation.
 The sequence fields to modify are:
 - `addr`: indicating the address of the write operation (CTRL).
 - `wr_data`: a byte array containing the value to be written (`{'h1}`).
- **`axi4lite_rd_data_deparam_seq axi4_master_0_seq_2`**: a basic read transaction to poll the STATUS (`0x8`) register. It's documentation can be found at: [\\$QVIP_HOME/docs/html/axi4/files/axi4/sequence_lib/stimuli/axi4lite/deparam/data_transfer/rd_data_deparam-svh.html](#)
 The sequence fields to modify are:
 - `addr`: indicating the address of the read operation (STATUS).
 - `rd_bytes`: amount of bytes to read (`0x4`).
 Should be repeated until the read value is different from 0.

- **axi4_incr_rd_deparam_seq axi4_master_0_seq_3**: sequence to read 64*8 bytes, corresponding to 64 real values.

The sequence fields to modify are:

- **addr**: corresponding to the starting address of the OUT_BLOCK, expressed in bytes (0x210), and later increased by the number of read bytes.
- **rd_bytes**: amount of bytes to read (8 each time).

The read bytes should be later concatenated into 64-bit words and transformed into real values (use SystemVerilog's \$bitstoreal() function).

This sequence should be executed (ie: call the start() method) 64 times, one for each value.

15. To run the simulation, a **make all** in the **top_dir/uvm_tb** should do.

Results Checking

16. Add the provided sv_library to the environment, which contains a DPI function that calculates the DCT
 - 16.1. Compile and generate the SystemVerilog library to be used as a DPI function. In the base directory of the repository, run **source dct.sh**, which should generate a **dct.so** file.
 - 16.2. For this, you should add the **-sv_lib** flag to the **vsim** command in the Makefile (ie: `UVM_OPTIONS = +UVM_TESTNAME=top_test_base +SEQ=top_example_vseq -sv_lib ../../dct`)
17. Import the DPI function to the sequence (**import "DPI-C" function int dct(input real in_block[8][8], output real out_block[8][8]);**)
18. Create a compare function in the sequence that calls the DPI function with the same values that were sent to the DUT and checks the DUT results against those of the DPI function.

EXTRA:

19. Create a UVM component (could be a **Scoreboard**) that receives the sent values to the DUT and the read values from the DUT. Instantiate and connect the component in the environment (uvm_tb/top_env.svh).
 - 19.1. Check which kind of port/ports you will need to create and connect in the corresponding connect phase.
 - 19.2. Hint: unless we create our own sequencer/driver that extends the VIP ones, we can not add a port to the existing sequencer/driver. There can be many workarounds to this issue, for example:
 - 19.2.1. A workaround to be able to write to an external port to the sequence, and have visibility from the environment and the sequence itself, would be to create a port in the existing configuration object **top_env_config**.
The top_vseq_base should have a pointer to this configuration object, assigned in the top_test_base.

In this way, we could connect the configuration object port to the scoreboard port in the environment's connect phase, and write transactions to the configuration object port from the sequence itself.

The port should be parameterized with a custom transaction that you should create. Newly created files should be added to the uvm_tb/top_pkg.sv for correct compilation.

- 19.2.2. Create a port in the environment, set it in the uvm_config_db and get it in the sequence. The environment should also connect this port to the scoreboard, while the sequence should write to it.
20. Replace the comparison checking in the sequence by sending input values and DUT results to the Scoreboard.

Deliver:

1. The modified uvm_tb/top_example_vseq.svh file.
2. A simulation waveform (could be 1+ screenshots) showing:
 - a. The transactions writing values to IN_BLOCK (addresses + values).
 - b. The transaction writing the CTRL register (address + value).
 - c. The transactions reading the STATUS register (address + read value).
 - d. The transactions reading values from OUT_BLOCK (addresses + read values).
3. A simulation log showing
 - a. **uvm_info** messages each time a value has been written/read and its corresponding address.
 - b. **uvm_info** messages showing compared values (DUT response vs. reference model/DPI output).
 - c. **uvm_error/uvm_fatal** messages if the comparison failed.
4. If you have created a Scoreboard:
 - a. The components you created.
 - b. The environment where your component was instantiated and connected.

Useful resources:

- Questa AXI4 VIP Manual: QUESTA-VIP_2021.3_2_RHELx86/Siemens/2021-22/RHELx86/QUESTA-VIP_2021.3_2/docs/html/axi4/index.html
- Qvip_configurator tutorial: <https://verificationacademy.com/topics/verification-ip/questa-verification-ip-configurator/>

Known issues:

- Issue: \$QUESTA_MVC_HOME environment variable
** Error: (vlog-7) Failed to open design unit file "/include/questa_mvc_svapi.sv" in read mode.

Solution: in aipcd-lab4/top_dir/uvm_tb/Makefile replace **\${QUESTA_MVC_HOME}** by **/tools/Siemens_EDA/questasim/2021.3/**

- Issue: 64-bit dct library

Error: (vsim-125) The shared library ../../dct.so being loaded was built on a 64-bit machine. A 64-bit shared library cannot be loaded in a 32-bit simulation. Please use compatible machines to build and load the library

Solution: in aipcd-lab4/top_dir/uvm_tb/Makefile add the flag **-64** after each vlog/vopt/vsim command